

Transferring Machine Learning Models to New Soft Pneumatic Actuators

Felix Tobias Schröter

Abstract—Morphological acoustic sensing using soft pneumatic actuators is a promising way to approach sensing in soft robotics. However, this entirely data-driven approach to sensing makes machine learning models fitted on data from one actuator susceptible to small differences in its hardware and resulting audio signal. This makes model transfer to new, unseen actuators difficult, even for hardware manufactured with the exact same process - motivating the need for a calibration method. In this work, we formalize calibration in acoustic sensing, collect new datasets on multiple actuators, and establish baselines for cross-actuator transferability and scaling with the number of actuators in the training set. We then use normalization techniques, affine transformations, and adversarial domain neural networks to create calibrated representations of audio samples. Finally, we evaluate these representations by fitting shallow machine learning models on them and show that both normalization and affine transformations increase the transferability to unseen actuators.

I. INTRODUCTION

We present a formalization of calibration in acoustic sensing, develop different approaches for calibrating sensor models to new actuators, and finally evaluate these approaches.

Soft robot hands such as the RBO are promising because they intrinsically offer mechanical compliance, making them safer for human-robot interaction, and can be produced at low cost using standard components [1]. The fingers (or soft pneumatic actuators) of the RBO hand are made of molded silicon and actuated with air pressure, which is why their sensorization is challenging, as classical approaches such as joint encoders cannot be used [2, 3]. Instead, the actuators shape can be treated as a sensor that can be read out, which is referred to as morphological sensing, or acoustic sensing if this is accomplished using audio. For this, a speaker is added inside the tip of the actuator which emits an active sound that is modulated by the hardware and recorded by a microphone inside the base. The resulting signal contains a rich representation of the actuator state and can be used to solve many different sensing tasks.

However, without more actuators in the training set, the resulting acoustic sensor models are poorly transferable — a sensor model trained on data from one actuator will not work well on a second one, even for actuators made with the same manufacturing process. This is because sensor models overfit to subtle differences in how the actuator’s hardware and materials modulate the sound, which has been described as one of the main limitations of acoustic sensing and shows the need for a calibration method.

The calibration of actuators can be seen as aligning the task-relevant feature spaces $\mathcal{X}_i$ of different actuators, which can be achieved by mapping to a shared representation

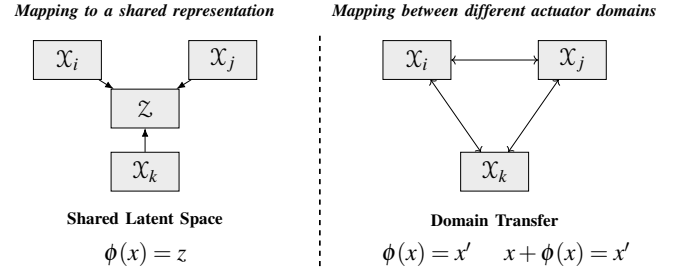


Fig. 1: Illustration of the two perspectives on calibration by domain adaption used in this work, for three example actuator i, j, k and their task-relevant feature spaces $\mathcal{X}_i, \mathcal{X}_j, \mathcal{X}_k$: it can be seen either as finding mappings $\phi_j : \mathcal{X}_j \rightarrow \mathcal{Z}$ to a shared latent space $\mathcal{Z}$, or mappings $\phi_{j,k} : \mathcal{X}_j \rightarrow \mathcal{X}_k$ between the feature spaces of the different actuators. Depending on the approach, we may only have a single function $\phi = \phi_{i|j|k}$, for example when using a neural network as a feature extractor.

space $\mathcal{Z}$ or between different feature spaces, as visualized in Figure 1. As these mappings can be derived from patterns in data, calibration and model transfer in adjacent research topics are often approached data-driven using transfer learning [4, 5].

In this work, we first formalize calibration in acoustic sensing and collect new datasets for a regression task. We then validate that the model for the task we defined is indeed not transferable without calibration and introduce data normalization as a baseline approach to improve transferability. We show that introducing additional actuators into the model training set leads to more transferable models. Then, we use an affine transformation to map between actuator feature spaces and show that this also enables better generalization to unseen actuators. Lastly, we introduce a neural network to learn actuator-invariant sample representations which is based on domain-adversarial neural networks (DANNs) and describe the problems we encountered with this approach that did not work in our setup [6].

II. RELATED WORK

The related work introduces where the transfer problem in acoustic sensing was described before, how calibration can be seen as domain adaption, and how it was approached through transfer learning in other fields in robotics. Furthermore, it introduces the concept that our neural network based feature extractor builds on.

A. Transferability of Acoustic Sensor Models

In their work introducing acoustic sensing for soft robotic actuators, Wall et al. show the poor transferability of the

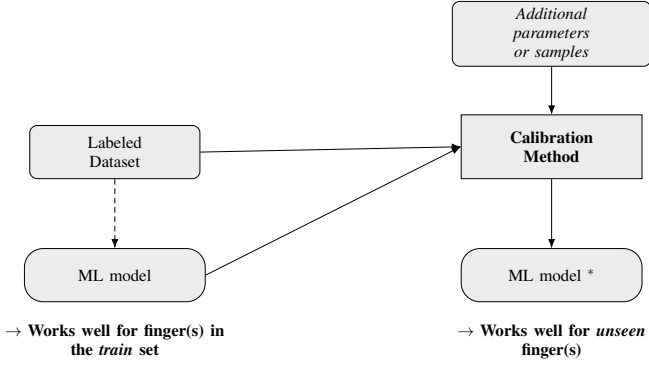


Fig. 2: Illustration of the calibration method: the calibration method generates a new ML model that works well on out-of-distribution actuators based on existing data, additional samples or parameters and an existing model.

sensor model and describe it as one of the key limitations of their sensing method [2, 3]. Furthermore, they suggest researching a calibration method and working with per-actuator sensor models until such a method is found.

B. Calibration by Domain Adaption

The domain adaption problem describes a transfer-learning situation in which, given a lot of labeled training data from a source domain and limited training data from a target domain, the goal is to learn a model that is applicable in the target domain [7]. In soft robotics, transfer learning has been used to reduce the number of samples needed to calibrate tactile shape and force sensors [4, 5], which can be viewed as data-driven calibration. This can also be combined with vision-based closed loop systems and artificial data from finite-element simulations [8], fitting generative models to create synthetic samples for downstream tasks [9], or using CycleGANs on simulated soft robots for cross-domain skill transfer [10]. Learning latent spaces across tasks and sensors using transformers is possible and improves transferability to new sensor task combinations, but requires large labeled datasets [11].

C. Domain Adversarial Neural Networks

DANNs are neural networks for domain adaption that learn domain-invariant latent representations through an adversarial classifier that has the target of predicting the domain from the latents. The adversarial objective of the domain classifier is implemented through a gradient reversal layer (GRL) that reverses the gradients of the classification loss on the backward pass. In addition to CycleGANs, DANNs have exhibited superior performance in robotic vision tasks [12].

III. CALIBRATION IN ACOUSTIC SENSING

We define calibration as a method that takes a dataset, a corresponding machine learning (ML) model, and some additional samples or parameters as input and produces a new ML model that is calibrated for unseen actuators (visualized in Figure 2), which is similar to the domain adaption problem (described in II-B).

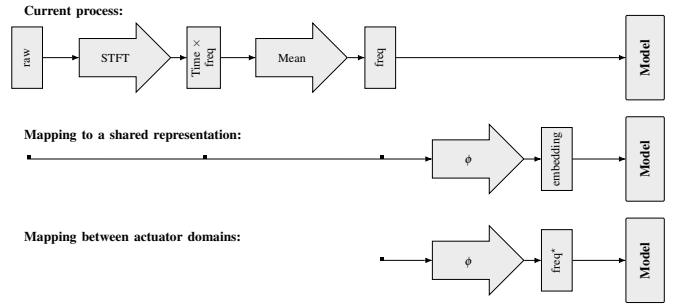


Fig. 3: SOT pipeline: current processing chain and two drop-in mapping variants used in this work. The *current process* consists of applying short-time fourier transform to the raw audio recordings, and then taking the mean per frequency over all time frames. The resulting frequency mean vectors are then used as input for a ML model. The first alternative (*mapping to a shared representation*) takes either the raw recordings, the STFT output, or the frequency mean vectors and maps them to an embedding, which is then passed to a model. The second alternative (*mapping between actuator domains*) takes the mean frequency vectors and maps them to a modified frequency vector which is more similar to the vectors the model was trained on.

The audio signal $s(t)$ that we record for acoustic sensor models is a modulation by the actuator $\Gamma_{actuator}$ of an active sound $\alpha(t)$ (e.g., a sweep) and outside noise $\eta(t)$. This modulation can be divided into modulation by the microphone Γ_{mic} , the embedded speaker $\Gamma_{speaker}$ and the actuator material $\Gamma_{material}$:

$$\begin{aligned} s(t) &= \Gamma_{actuator} [\alpha(t), \eta(t)] \\ &= \Gamma_{mic} [\Gamma_{material} [\Gamma_{speaker} [\alpha(t)]] + \Gamma_{material} [\eta(t)]] \end{aligned} \quad (1)$$

The modulation of the signal by the material $\Gamma_{material}$ changes as the actuator is stretched, manipulated, or the conditions at its boundary change. The ML models we train on samples from $s(t)$ essentially capture the changes in the modulation when we assume constant $\alpha(t)$ and $\eta(t)$. These changes in the modulation are non-trivial and actuator-specific, which is why we try to estimate the difference across actuators from samples and then create transformations for the data to train more transferable models. This can either mean finding a mapping that removes the actuator-specific part of the signal or approximates the differences in signal between actuators and maps between their feature spaces, as visualized in Figure 1.

The state-of-the art process (Figure 3) for creating new models is to apply short-time fourier transform (STFT) to the raw audio recordings, take the mean of each frequency bin over all time windows, and finally fit a ML model such as a k-nearest neighbors classifier (KNN), as outlined by Wall et al. [3], and implemented in the corresponding acoustic sensing starter kit¹. Our approaches extend this process by applying the two views (Figure 1), either by mapping one of the representations (raw or mean frequencies) to a shared embedding space or by transforming the mean frequency

¹https://git.tu-berlin.de/rbo/robotics/acoustic_sensing_starter_kit



Fig. 4: Picture of the recording setup used to collect the data for the regression task. The ruler is used to measure the distance from the base and the actuator is tapped from the top using a small wooden stick.

vectors to make them more similar to data collected on another actuator.

IV. EXPERIMENTAL SETUP

A. Tasks and Datasets

We recorded data for two main tasks on five different actuators:

- A contact localization *regression* task where we touched the actuator from the top in 0.5 cm steps, starting from the base using a thin wooden stick. With 30 one second recordings for each of the 20 different labels $\{0, 0.5, \dots, 9.0, \text{no touch}\}$, a total of 600 samples per actuator were recorded for this task.
- An *unlabeled* dataset that consists of a consecutive 10 minute recording per actuator where it was excited by tapping, pinching, brushing, and bending it, both with a pencil and by hand.

The data for both tasks were recorded using a linear sweep from 20 to 20000 Hertz over one second, consecutively repeating the sweep for the recording length of the unlabeled dataset. Figure 4 shows the recording setup used to collect the regression dataset and Table I shows all available datasets.

V. METHODS

A. Normalization

Different normalization methods worked as a baseline approach to equalize actuator-specific modulations, as described in Section III. Volume levels and peaks are absolute in the unnormalized signal; normalizing the samples transforms these absolute differences into relative ones and makes them more comparable across actuators. We compared unnormalized audio, with z-score (Eq. 2a), as well as min-

max (Eq. 2b) normalizations of the data,

$$x'_{act,freq} = \frac{x_{act,freq} - \mu_{act,freq}}{\sigma_{act,freq}} \quad (2a)$$

$$x'_{act,freq} = \frac{x_{act,freq} - x_{act,freq}^{\min}}{x_{act,freq}^{\max} - x_{act,freq}^{\min}} \quad (2b)$$

separately applied to each frequency bin on each actuator.

B. Affine Transformation

Affine transformations can rescale and shift samples recorded on one actuator so that they have more similar properties to samples recorded on another actuator, e.g. adapting to loudness levels or shifts in the frequency space. We define an affine transformation as a mapping between the feature spaces $\mathcal{X}_l$ and $\mathcal{X}_m$ of two actuators l and m , such that approximately:

$$\forall \mathbf{x} \in \mathcal{X}_l : \quad \mathbf{x}' = \mathbf{A}_{l,m} \mathbf{x} + \mathbf{b}_{l,m} \in \mathcal{X}_m. \quad (3)$$

We estimate $\mathbf{A}_{l,m}$ and $\mathbf{b}_{l,m}$ by solving a least-squares problem on the mean feature vectors per-label $\bar{\mathbf{x}}_y$ and $\bar{\mathbf{x}}'_y$, over all labels $y \in \mathcal{Y}$,

$$\min_{\mathbf{A}_{l,m}, \mathbf{b}_{l,m}} \sum_{y \in \mathcal{Y}} \|\mathbf{A}_{l,m} \bar{\mathbf{x}}_y + \mathbf{b}_{l,m} - \bar{\mathbf{x}}'_y\|_2^2, \quad (4)$$

to adjust the affine mapping to the task. These affine mappings can be rewritten in a residual form, i.e. $x' = x + \phi(x)$, implying that they fit the differences in the mean frequencies between two actuators.

C. Domain Adversarial Neural Network

Our implementation of the DANN consists of the following components, which are also visualized in Figure 5:

- An *encoder* network f_θ , a fully connected multi-layer perceptron (MLP) that maps input features to a latent space: $\mathbf{z} = f_\theta(\mathbf{x})$.
- A fully-connected *reconstruction head (decoder)* g_ϕ that maps latents back to the input space: $\hat{\mathbf{x}} = g_\phi(\mathbf{z})$.
- A *regression head* h_ψ (MLP) that predicts the target from the latents: $\hat{y} = h_\psi(\mathbf{z})$.
- An *adversarial domain classifier* d_ω that takes a subset of the latents $\mathbf{z}_d \subseteq \mathbf{z}$ and predicts the actuator/domain: $\hat{s} = d_\omega(\mathbf{z}_d)$.
- A GRL $\mathcal{R}$ between the latents and the domain classifier, which is the identity in the forward pass but scales and reverses the gradients: $\mathcal{R}(\mathbf{z}) = \mathbf{z}$ and $\frac{\partial \mathcal{R}}{\partial \mathbf{z}} = -\mathbf{I}$ [6].

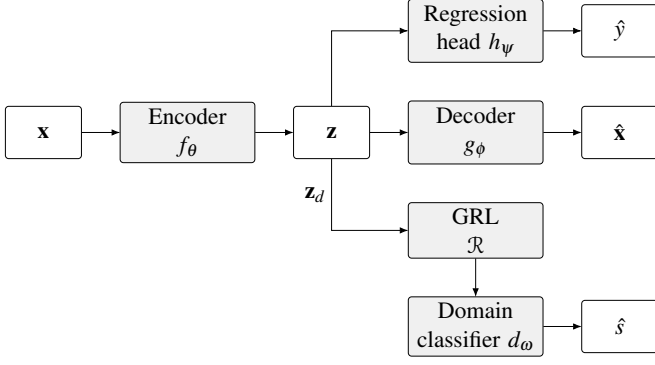


Fig. 5: Visualization of the DANN architecture: samples x are passed to the encoder f_θ which creates the latents z . The latents are passed to the regression head h_ψ that predicts a regression target $\hat{y}$, the decoder g_ϕ that reconstructs the input $\hat{x}$ and to the adversarial domain classifier d_ω that predicts the actuator the sample was recorded on $\hat{s}$. The GRL reverses the gradients on the backward pass.

The loss function to train this network consists of multiple weighted targets: a regression target (Eq. 5) and a reconstruction target (Eq. 6), which are both mean squared error (MSE) losses, a cross-entropy (CE) loss for the adversarial classifier target (Eq. 8), as well as a triplet loss directly applied to the latent variables (Eq. 7)[13]. We define the individual loss terms as follows:

$$L_{\text{reg}}(\theta, \psi) = \text{MSE}(y, \hat{y}), \quad (5)$$

$$L_{\text{rec}}(\theta, \phi) = \text{MSE}(x, \hat{x}), \quad (6)$$

$$L_{\text{tri}}(\theta) = L_{\text{tri}}(z), \quad (7)$$

$$L_{\text{dom}}(\theta, \omega) = \text{CE}(s, \hat{s}). \quad (8)$$

which results in the overall target:

$$\min_{\theta, \phi, \psi} \max_{\omega} L = \alpha L_{\text{reg}} + \beta L_{\text{rec}} + \gamma L_{\text{tri}} - \delta L_{\text{dom}}. \quad (9)$$

We train the model both on the regression and the unlabeled datasets. All samples have an auxiliary `has_regression_label` attribute, and the regression loss is set to 0 for a sample if no such label is available.

Building on this overall architecture, we used different configurations:

- A simple autoencoder, consisting only of the encoder and the decoder parts (f_θ and g_ϕ)
- The autoencoder plus the regression head (h_ψ)
- Adding only the GRL and domain classifiers to the autoencoder ($\mathcal{R}$ and d_ω)
- The complete architecture with all of the components
- Varying hyperparameters such as a number of layers or latent size
- Training on the unlabeled datasets, regression datasets, or both

D. Transferability Evaluation

For our evaluations, we train three different regression ML models (LGBMRegressor [14] and KNeighborsRegressor, Ridge [15]) on a train split of the representations obtained from the data recorded on one

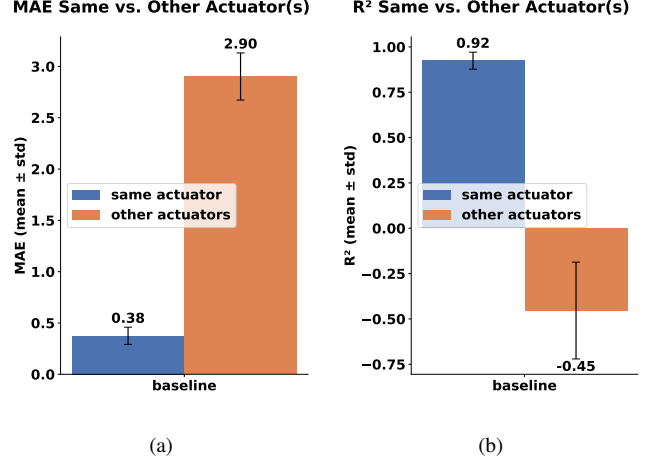


Fig. 6: Transferability without calibration: The MAE (a) and R^2 scores (b) of uncalibrated models on data from the actuator in the training set and on data recorded on unseen actuators. Both the MAE and the R^2 scores are significantly worse on the out-of-distribution actuators than on the actuator the model was trained on, which shows that the models are not transferable without calibration.

actuator. We then score the predictions on the test split of the data and the representations obtained from the data recorded on the other actuators using coefficient of determination (R^2) and mean absolute error (MAE) as scores. An R^2 score of one shows that the model perfectly predicted the labels, zero corresponds to the model performing equal to a mean predictor, and negative values mean worse performance than the mean predictor. We randomize the split and repeat it using 50 different random seeds, we then take the average per model and actuator, and keep only the best performing model by MAE on the other actuators.

VI. RESULTS

A. Transferability without Calibration

As a baseline for transferability, we evaluated a ML model trained on data for the regression task and obtained through the steps of the current state of the art process (described in III) without any additional calibration. For the regression task, this model achieves a mean test MAE of 0.38 cm for data recorded on the same actuator as the model training data. For the other out-of-distribution (or unseen) actuators, the mean MAE increases to 2.90 cm, as Figure 6a shows. The mean R^2 decreases from 0.92 to -0.45 when moving from in-distribution to out-of-distribution data (Figure 6b), showing that the model performs worse on the unseen actuators than a mean predictor. This also establishes that the models are not transferable without calibration.

B. Effect of Normalization

When introducing min-max normalization before the modeling step, the mean MAE decreases to 0.32 cm for data from the same actuator and to 2.75 cm for unseen actuators. For the z-score normalization the mean MAE on the same

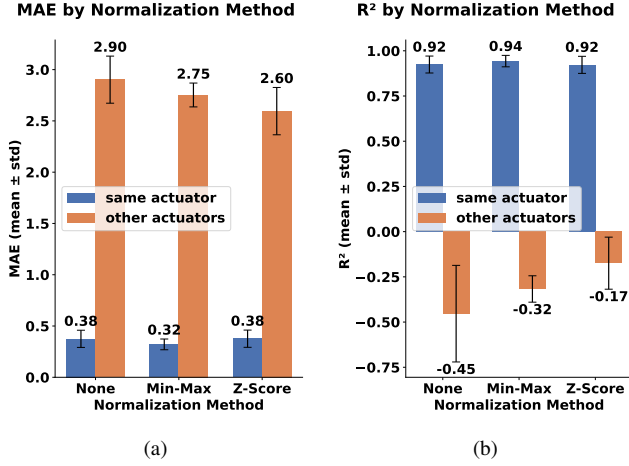


Fig. 7: Normalization: the MAE (a) and R^2 (b) of different normalization methods (no normalization, min-max, and z-score) evaluated on the actuator in the train set and out-of-distribution actuators. With both normalization methods, the MAE and R^2 show better generalizability to unseen actuators compared to no normalisation, which indicates that absolute differences in the signal such as volume are likely more aligned.

actuator stays constant, but decreases to 2.60 cm for data from unseen actuators, as Figure 7a shows. The R^2 stays almost constant for the in-distribution actuators, but increases to -0.32 for min-max and -0.17 for z-score normalizations for the unseen actuators (Figure 7b), therefore still performing worse than a mean predictor on the other actuators. This also indicates that the introduction of normalization aligns volumes and increases the transferability.

C. Increasing the Number of Actuators in the Train Set

Adding more actuators to the training dataset reduces the out-of-distribution mean MAE to 2.40 cm for training on two actuators and to 2.29 cm for training on three actuators. The in-distribution mean MAE increases to 0.54 cm for two actuators in the train set and to 0.62 cm for three actuators, as shown in Figure 8a. The mean R^2 on the data from the actuators in the training set remains constant at 0.92 when one additional actuator is added and decreases to 0.90 with a train set size of three (Figure 8b). The out-of-distribution mean R^2 increases to 0.01 for one additional actuator and 0.09 for the addition of two actuators to the training set. This shows that adding additional actuators to the train set makes the resulting model perform better than a mean predictor on unseen actuators. It also shows that increasing the number of actuators in the train set is a way to create more transferable models.

D. Effect of Affine Transformation

Projecting the (z-score normalized) data onto the actuator feature space that the ML model was fitted to decreases the mean MAE to 2.25 cm for out-of-distribution actuators, but increases it to 0.62 cm for the actuator in the train set (Figure 9a). The mean R^2 increases to 0.01 for unseen

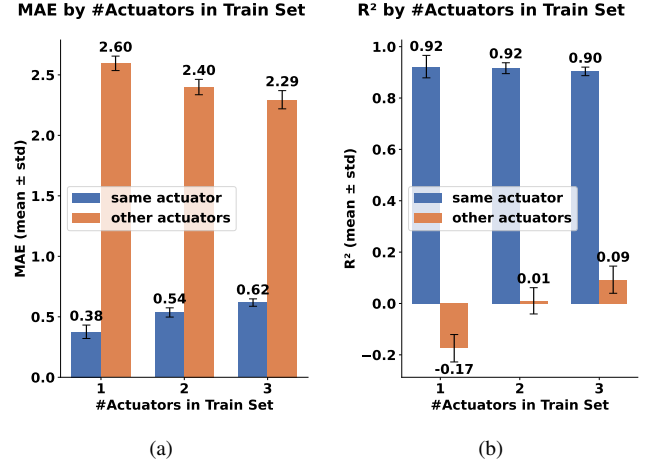


Fig. 8: *Finger Scaling Law*: the MAE (a) and R^2 (b) of different training set sizes (1-3 actuators), evaluated on in-distribution and out-of-distribution data. With more actuators in the train set, the transferability increases and with three actuators the resulting models are better than a mean predictor, as the MAE and R^2 scores show.

actuators and decreases to 0.86 for the actuator in the training set, therefore showing an improvement compared to the z-score normalization and on-par performance with a mean predictor. This indicates that the affine transformation aligns feature spaces beyond the alignment by normalization.

E. Deep-Learning Based Approach

We tried many different configurations of the DANN architecture outlined in section V-C, but the reconstruction and regression tasks did not produce embeddings with enough task specific information and were also not able to produce embeddings that are actuator-invariant. The adversarial objective of the domain classifier and the min-max form of the overall target (Eq. 9) led to unstable training behavior and embedding collapse. Introducing the triplet loss led to further oscillations of the loss function. Within our setup and the experiments we ran, we were unable to make this approach work.

VII. DISCUSSION

Our results show that using the current process for modeling in acoustic sensing does not create models that are inherently transferable to actuators not in the training set. This is likely due to a hardware-specific modulation $\Gamma_{actuator}$ (Eq. 1) of the signal to which the ML models are overfitting to.

When we normalize or standardize in the feature space, the transferability improves, which is likely because the absolute differences in the signal are removed and, therefore, equalized. The z-score norm is less sensitive to outliers and single values in the data, which could explain why it showed a larger positive effect than the min-max norm.

Increasing the number of actuators in the train set improves the transferability of the resulting model. This indicates that there is a shared (task-specific) feature space

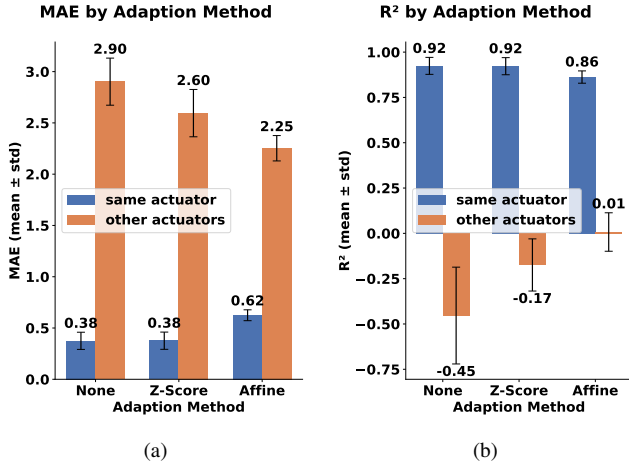


Fig. 9: Affine transformation: the MAE (a) and R^2 (b) of the different adaption methods (no normalization, z-score, and affine transformation) evaluated on seen and unseen actuators. Mapping the samples to the feature space of the actuator that the model was fitted on using affine transformations (after applying the z-score norm) leads to better R^2 and MAE scores than only using z-score. This shows that the affine transformations can align the task relevant feature spaces beyond what the normalization can do.

between the different actuators and that learning actuator-invariant models might require training on multiple actuators to generalize.

Projecting data to the feature space of the actuator that a model was fitted on using the affine transform also improved the transferability to a similar extent to adding one additional actuator to the training set. However, this method was still barely able to outperform a mean predictor, as the R^2 score shows. Most likely, this is because the affine transformation is a linear mapping with limited expressiveness. Therefore, this result does not present a standalone calibration method yet, but rather should be seen as initial evidence that residual models are a promising approach to calibration and that non-linearity needs to be introduced in future research on this approach.

Lastly, we learned that adversarial architectures are challenging to train and did not produce meaningful representations in all the configurations we tried in our setup.

The results are limited by the low total number of actuators that were used to record the data for this work, the dissimilarity of some of the actuators, and the fact that we evaluated on a single contact localization regression task.

In our future research, we would therefore like to record data on more actuators, re-record on a set of even more similar actuators, and define a second regression task. We also want to further explore the general direction of the affine transformation, and develop a non-linear residual model for learning the domain transfer.

VIII. CONCLUSION

In this work, we showed how calibration can be viewed and formalized in the acoustic sensing domain and developed some approaches to showcase initial steps towards a

TABLE I: Availability of datasets for the different tasks by actuator.

	Classification	Regression	Regression 2	Unsupervised
Actuator 1	✓	✓	✓	✓
Actuator 2	✓	✓	✓	✓
Actuator 3	✓	broken	broken	broken
Actuator 4 (not recorded)		✓	✓	✓
Actuator 5 (not recorded)		✓	✓	✓

calibration method. Furthermore, our experiments indicate that there is a task-specific shared feature or latent space between actuators. The normalization methods and affine transformation showed that we can transform the recorded data so that these feature spaces are more aligned, which is why we think that further research of domain transfer with residual models is promising. In general, calibration in acoustic sensing remains challenging and unsolved, but our work shows that there are promising approaches that need to be explored further.

APPENDIX

A. Datasets and Code Availability

All of our experimental data that were recorded, as well as the code used to implement the different modeling approaches and create the plots, are available on Gitlab². A total of 5160 labeled samples and 40 min of unlabeled excitations were recorded for this project. In addition to the tasks described in Section IV-A, we recorded data for two more tasks:

- A contact-localization *classification* task in which the actuator was tapped from the top during the duration of a one second sweep, with labels {base,middle,tip,none}. We recorded 30 samples per label and actuator for this dataset.
- Another *regression* task with the same labels as the other one, but where we first started the sweep and only then tapped the actuator.

One of the actuators broke during the recording, and the availability of the different tasks can be seen in Table I.

B. Usage of AI Tools

We used *ChatGPT*³ to identify relevant research areas during the exploratory phases of this research project and generate code snippets from natural language, which is also documented in the code base. Furthermore, we used the integrated *Overleaf AI assistant*⁴ to adapt formatting, turn screenshots from presentations and descriptions into latex graphics, create formatted equations from descriptions, and get pointers on text readability.

²https://git.tu-berlin.de/felix_schroeter/transferring_ml_models_to_new_actuators

³<https://chatgpt.com/>

⁴<https://www.overleaf.com/about/ai-features>

REFERENCES

- [1] R. Deimel, M. Radke, and O. Brock, "Mass control of pneumatic soft continuum actuators with commodity components," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Daejeon, South Korea: IEEE, Oct. 2016, pp. 774–779. [Online]. Available: <http://ieeexplore.ieee.org/document/7759139/>
- [2] V. Wall, G. Zöller, and O. Brock, "Passive and active acoustic sensing for soft pneumatic actuators," *The International Journal of Robotics Research*, vol. 42, no. 3, pp. 108–122, Mar. 2023. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/02783649231168954>
- [3] V. Wall, "Morphological sensing for soft pneumatic actuators based on acoustics and strain," 2024. [Online]. Available: <https://depositonce.tu-berlin.de/handle/11303/21090>
- [4] W. K. Do, B. Jurewicz, and M. Kennedy, "DenseTact 2.0: Optical Tactile Sensor for Shape and Force Reconstruction," Mar. 2023, arXiv:2209.10122 [cs]. [Online]. Available: <http://arxiv.org/abs/2209.10122>
- [5] N. Masoumi, A. C. Ramos, T. Torkaman, L. S. Feldman, J. Barralet, J. Dargahi, and A. Hooshier, "Embedded Force Sensor for Soft Robots With Deep Transformation Calibration," *IEEE Transactions on Medical Robotics and Bionics*, vol. 6, no. 4, pp. 1363–1374, Nov. 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10715689>
- [6] Y. Gani, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-Adversarial Training of Neural Networks," *Journal of Machine Learning Research*, May 2015.
- [7] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Machine Learning*, vol. 79, no. 1-2, pp. 151–175, May 2010. [Online]. Available: <http://link.springer.com/10.1007/s10994-009-5152-4>
- [8] M. Wiese, G. Runge-Borchert, B.-H. Cao, and A. Raatz, "Transfer learning for accurate modeling and control of soft actuators," in *2021 IEEE 4th International Conference on Soft Robotics (RoboSoft)*, Apr. 2021, pp. 51–57. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9479300/citations>
- [9] S. Sapai, J. Y. Loo, Z. Y. Ding, C. P. Tan, V. M. Baskaran, and S. G. Nurzaman, "A Deep Learning Framework for Soft Robots with Synthetic Data," *Soft Robotics*, vol. 10, no. 6, pp. 1224–1240, Dec. 2023. [Online]. Available: <https://www.liebertpub.com/doi/10.1089/soro.2022.0188>
- [10] N. Kushawaha, C. Alessi, L. Fruzzetti, and E. Falotico, "Domain Translation of a Soft Robotic Arm using Conditional Cycle Generative Adversarial Network," 2025, version Number: 1. [Online]. Available: <https://arxiv.org/abs/2508.14100>
- [11] J. Zhao, Y. Ma, L. Wang, and E. H. Adelson, "Transferable Tactile Transformers for Representation Learning Across Diverse Sensors and Tasks," Oct. 2024, arXiv:2406.13640 [cs]. [Online]. Available: <http://arxiv.org/abs/2406.13640>
- [12] M. H. Tanveer, Z. Fatima, S. Zardari, and D. Guerra-Zubiaga, "An In-Depth Analysis of Domain Adaptation in Computer and Robotic Vision," *Applied Sciences*, vol. 13, no. 23, p. 12823, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/23/12823>
- [13] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," 2015, pp. 815–823. [Online]. Available: https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Schroff_FaceNet_A_Unified_2015_CVPR_paper.html
- [14] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: <http://jmlr.org/papers/v12/pedregosa11a.html>